Resting order book: performance analysis

Rushi Jadhav | roll 10144 | rushi.24bcs10144@sst.scaler.com | individual submission | 17 October 2026

A uses tree price levels and hashed orders. B uses sorted level vectors and an ordered ID tree. C combines fixed tick arrays, occupancy words, cached best prices and packed orders indexed by hashed ID. All are C++17 resting books: no matching or trades. Integer cents avoid floating-price truncation; all 10,000 valid ticks are supported.

Experiment and results

Local i7-14650HX, Linux 7.2.9-2-cachyos, GCC 16.2.1, -O2 -DNDEBUG -g -fno-omit-frame-pointer; CPU 0 (performance core), existing powersave governor unchanged. Seed 42 generated 1,000,000 primary operations: 699,318 adds (350,376 buys), 200,311 successful live cancels, 100,371 BBO queries, plus 20,000 top-10 depth calls. Trace hash 14416990210762577260 and checksum 11460434319816070843 matched across books. Independent-reference, Debug, ASan/UBSan and clean-build verification passed.

Metric	A	B	C
Total time (ms)	752.665	959.002	374.992
ns / add	647.651	682.224	199.407
ns / cancel	900.632	1,814.171	579.639
ns / BBO	119.974	110.162	106.469
IPC	1.196	1.298	1.845
Branch misses (%)	4.098	3.716	1.007
L1-dcache-load-misses	12,925,929	19,726,253	4,226,036
Cache misses	7,250,501	10,701,205	5,485,506
Page faults	8,429	11,535	6,615

Timing rows are independent medians of five rotated, sequential, consistently warmed fresh-book runs. Generation, construction/destruction and output are excluded; total includes per-call clocks, checksum folding and mandatory depth queries. No book receives workload-sized reserves. Counter rows are full-process perf stat -r 5 reported means, including generation and teardown. IPC = mean instructions / mean cycles; branch % = 100 x mean misses / mean branches. Scopes differ; counters are not book-only measurements.

1. Winners by operation

C wins measured add, cancel and BBO averages and is 2.01x faster overall than A (2.56x than B). Direct tick indexing removes price-tree searches and vector-level shifting. Packed order storage plus hash-to-slot cancellation avoids B's random ID-tree traversal; it still requires hash lookups and a moved-slot repair. A's sampled call graph attributes 37.92% inclusive to add_order, including price-tree searches. B attributes 23.85% inclusive to cancel_order, consistent with expensive ID lookup. These are sampled process shares, not exact operation durations.

C's BBO advantage over B is only 3.693 ns; all BBO averages include clock/dispatch overhead and do not measure isolated book-access latency. Cached indices and vector endpoints are both O(1); A also has O(1) endpoints but more indirection. Five run averages do not establish tail guarantees or statistical significance for such a small difference.

2. Branch prediction

B has the most absolute misses: 11.457 million, versus A's 8.574 million and C's 1.838 million. A has the highest miss percentage (4.098%); B executes more branches (308.336 million versus A's 209.215 million). Random price-tree paths in A and random cancellation paths in B's ID tree plausibly cause hard-to-predict left/right decisions. C removes those tree traversals. This explanation fits the counters and source, but shared generator/timer branches mean these totals do not isolate each book's branches. B's higher IPC than A does not make it faster: it executes more instructions overall.

3. Cache behavior

B has the most L1 and generic cache misses: 19.726 and 10.701 million. Its contiguous level vector does not remove pointer chasing through roughly half a million live ID-tree nodes. C has 4.226 million L1 misses and 5.486 million generic cache misses; fixed quantity arrays occupy 80,000 bytes plus occupancy/cache metadata, and orders are packed. Hash nodes and random slot access still create misses. A's tree/hash nodes also scatter accesses. The observed ranking supports locality as a contributor, not a proof that every difference is a cache effect; generic cache misses and L1 misses are different events.

4. Page faults and allocation

B has the most reported faults (11,535), followed by A (8,429), then C (6,615). B's per-order tree nodes and allocator activity are plausible contributors. All counts also include the common generated trace and process startup; C still allocates hash nodes and occasionally grows/copies its order vector. Faults indicate page population/access behavior, not total allocated bytes or resident memory. No memory-usage experiment or allocation-free guarantee is claimed.

5. Production choice

For this fixed price domain I would develop C further: it has the best measured latency, direct level access, a bounded best-removal occupancy scan and compact order storage. Its fixed level footprint is predictable, but hash rehashes, node allocation and vector growth can cause spikes; retained capacity follows peak live orders. Worst-case cancellation includes hash lookup and an occupancy scan, not guaranteed constant latency. Standard containers and a small slot-repair invariant keep it maintainable, though A is simpler if the price domain changes. Production acceptance needs realistic capacity limits, memory measurements and p99/p999 evidence, not this average-only ranking.

6. Next experiments and profiling limits

C's original call graph spends 29.64% inclusive in generation and 17.90% in clock_gettime. A separately labelled bulk replay removes per-call clocks while keeping the same trace, depth queries and checksum; its five-run median totals are A 615.430 ms, B 832.977 ms, C 224.511 ms. These show material instrumentation cost; they are not pooled into the main table. Bulk profiles retain substantial generation/allocator work, so neither mode isolates book-only counters. Follow-up work should test realistic capacity reservation/pools, reusable depth output buffers, C++20 standard bit-position operations, sparse/churning levels, nonsequential IDs and tail latency under growth pressure.

All requested performance-core counters and A/B/C call-graph recordings succeeded, with zero lost samples. Events are user-only; full-event core counts show no reduced running percentages. Unused atom-PMU entries are not counted and its L1 event is unsupported; they are excluded, never treated as zeros. Call graphs contain unresolved outer startup frames, so interpretation uses resolved book/generator/timer symbols. Raw commands, split-event checks, counters, runs and reports are in results/; methodology and reproduction commands are in README.md.